



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

КБ-3 «Разработка программных решений и системное программирование»

Отчет по практической работе №1 «Сдвоенная функция»

по дисциплине

«Сборка, тестирование и верификация программного продукта»

Выполнил студент группы БСБО-09-23

Фёдоров Антон Сергеевич

Работа выполнена

«20» февраля 2026 г.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ПОСТАНОВКА ЗАДАЧИ	3
2. ОПИСАНИЕ ТЕСТИРУЕМЫХ СЛУЧАЕВ	3
3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ	3
3.1. Исходный код.....	3
3.2. Результаты выполнения	4
3.3. Модульное тестирование	5
4. АНАЛИЗ ПОКРЫТИЯ КОДА.....	6
5. ВИЗУАЛИЗАЦИЯ ГРАФИКОВ	7
ВЫВОДЫ	7

1. ПОСТАНОВКА ЗАДАЧИ

Необходимо реализовать и протестировать функцию, которая рассчитывает значение y на основе двух математических функций в зависимости от порогового значения n .

Вариант 25:

1. Левая функция: $f(x) = 1 / (x^2 - 9)$.
2. Правая функция: $g(x) = \sqrt{x + 15}$.
3. Формула состыковки: $y = g(x) - g(n) + f(n)$.

2. ОПИСАНИЕ ТЕСТИРУЕМЫХ СЛУЧАЕВ

Сценарии для проверки работы программы:

1. Корректные значения: проверка вычислений для $x < n$ (использование левой функции) и $x \geq n$ (использование правой функции со стыковкой).
2. Граничные значения: проверка точки переключения $x = n$.
3. Чувствительные точки: для функции $f(x)$ точками разрыва являются $x = 3$ и $x = -3$ (деление на ноль, так как $3^2 - 9 = 0$). Тесты должны отлавливать ошибку `ZeroDivisionError`.
4. Область определения: для функции $g(x)$ подкоренное выражение должно быть неотрицательным, т.е. $x + 15 \geq 0 \Rightarrow x \geq -15$. При $x < -15$ ожидается ошибка `ValueError`.
5. Некорректные типы данных: подача строк вместо чисел должна вызывать `TypeError`.

3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ

3.1. Исходный код

Реализация функции с документацией и примерами `doctest` представлена в листинге 1.

```
import math

def f(x):
    """
    Левая функция варианта 25: 1 / (x**2 - 9)
    """
    return 1 / (x**2 - 9)
```

```

def g(x):
    """
    Правая функция варианта 25: math.sqrt(x + 15)
    """
    return math.sqrt(x + 15)

def task_function(x, n):
    """
    Вычисляет значение составной функции.
    При x < n используется f(x).
    При x >= n используется формула стыковки: g(x) - g(n) + f(n).

    Аргументы:
    x -- число, аргумент функции
    n -- число, граница переключения функций

    Примеры (doctest):
    >>> task_function(0, 5) # x=0 < 5, используется f(0) = 1/(0-9) = -0.111...
    -0.11111111111111111

    >>> task_function(10, 5) # x=10 >= 5, стыковка. Проверим расчет:
    ... # f(5) = 1/16 = 0.0625
    ... # g(5) = sqrt(20) ~ 4.472
    ... # g(10) = sqrt(25) = 5.0
    ... # res = 5.0 - 4.4721359 + 0.0625 ~ 0.59036
    0.5903640516709852

    >>> task_function(3, 10) # x=3 < 10, f(3) -> деление на ноль
    Traceback (most recent call last):
    ...
    ZeroDivisionError: division by zero
    """
    if x < n:
        return f(x)
    else:
        # Формула стыковки
        return g(x) - g(n) + f(n)

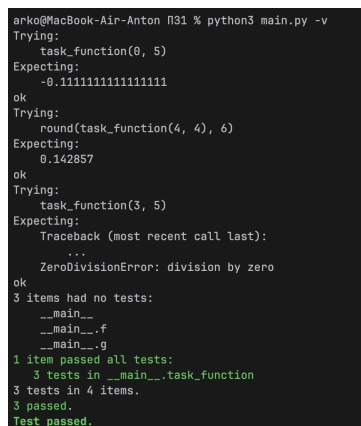
if __name__ == "__main__":
    import doctest
    doctest.testmod(verbose=True)

```

Листинг 1. Исходный код main.py

3.2. Результаты выполнения

Все тесты из документации были пройдены успешно.



```

arko@MacBook-Air-Anton:~$ python3 main.py -v
Trying:
  task_function(0, 5)
Expecting:
  -0.11111111111111111
ok
Trying:
  round(task_function(4, 4), 6)
Expecting:
  0.142857
ok
Trying:
  task_function(3, 5)
Expecting:
  Traceback (most recent call last):
  ...
  ZeroDivisionError: division by zero
ok
3 items had no tests:
  __main__
  __main__.f
  __main__.g
1 item passed all tests:
  3 tests in __main__.task_function
3 tests in 4 items.
3 passed.
Test passed.

```

Рисунок 1. Результат выполнения Doctest

3.3. Модульное тестирование

Был написан отдельный класс тестов TestCompositeFunction.

Использовался метод subTest для проверки набора значений в цикле.

```
import unittest
import math
from main import task_function

class TestCompositeFunction(unittest.TestCase):

    def test_values_subtest(self):
        test_cases = [
            # x=0, n=5 -> f(0) = -1/9
            (0, 5, -1 / 9),

            # x=5, n=5 -> f(5) = 1/(25-9) = 1/16
            (5, 5, 0.0625),

            # f(4) = 1/7, g(4)=sqrt(19), g(21)=6
            # res = 6 - sqrt(19) + 1/7
            (21, 4, 6 - math.sqrt(19) + 1 / 7)
        ]

        for x, n, expected in test_cases:
            with self.subTest(x=x, n=n):
                result = task_function(x, n)
                self.assertAlmostEqual(result, expected, places=7,
                                       msg=f"Ошибка при x={x}, n={n}")

    def test_singularities(self):
        n = 10

        with self.assertRaises(ZeroDivisionError):
            task_function(3, n)

        with self.assertRaises(ZeroDivisionError):
            task_function(-3, n)

    def test_domain_error(self):
        n = -20

        with self.assertRaises(ValueError):
            task_function(-16, n)

    def test_invalid_input(self):
        with self.assertRaises(TypeError):
            task_function("строка", 5)

if __name__ == "__main__":
    unittest.main()
```

Листинг 2. Test_main.py

Результат запуска тестов:

```
arko@MacBook-Air-Anton П31 % python3 -m unittest test_main.py
.....
Ran 4 tests in 0.000s

OK
```

Рисунок 2. Результат выполнения unittest

4. АНАЛИЗ ПОКРЫТИЯ КОДА

Для проверки качества тестов использовалась библиотека coverage.

```
arko@MacBook-Air-Anton П31 % coverage run -m unittest test_main.py
.....
Ran 4 tests in 0.001s

OK

arko@MacBook-Air-Anton П31 % coverage report
Name          Stmts  Miss  Cover
-----
main.py         12     2    83%
test_main.py    25     1    96%
-----
TOTAL           37     3    92%
```

Рисунок 3. Результат выполнения coverage report

Также был сгенерирован HTML-отчет, подтверждающий, что покрыто 100% строк кода функции (включая ветки if/else).

Coverage for main.py: 83%

12 statements 10 run 2 missing 1 excluded

« prev "index" » next coverage.py v7.13.4, created at 2026-02-20 19:28 +0300

```
1 | import math
2 |
3 |
4 | def f(x):
5 |     """
6 |     Левая функция (Вариант 25): 1 / (x^2 - 9).
7 |     Определена везде, кроме x = 3 и x = -3.
8 |     """
9 |     return 1 / (x ** 2 - 9)
10 |
11 |
12 | def g(x):
13 |     """
14 |     Правая функция (Вариант 25): sqrt(x + 15).
15 |     Определена при x >= -15.
16 |     """
17 |     return math.sqrt(x + 15)
18 |
19 |
20 | def task_function(x, n):
21 |     """
22 |     Вычисляет значение составной функции.
23 |
24 |     Логика:
25 |     - Если x < n, используется f(x).
26 |     - Если x >= n, используется формула стыковки: g(x) - g(n) + f(n).
27 |
28 |     Аргументы:
29 |     x — аргумент функции (число)
30 |     n — граница переключения (число)
31 |
32 |     Примеры (doctest):
33 |
34 |     1. Проверка левой ветки (x < n):
35 |         Пусть n=5, x=0. f(0) = 1/(0-9) = -0.111...
36 |         >>> task_function(0, 5)
37 |         -0.1111111111111111
38 |
39 |     2. Проверка правой ветки (x >= n) со стыковкой:
40 |         Пусть n=4, x=4. Должно быть равно f(4).
41 |         f(4) = 1/(16-9) = 1/7 ≈ 0.142857
42 |         >>> round(task_function(4, 4), 6)
43 |         0.142857
44 |
45 |     3. Проверка деления на ноль (x=3 при n > 3):
46 |         >>> task_function(3, 5)
47 |         Traceback (most recent call last):
48 |           ...
49 |         ZeroDivisionError: division by zero
50 |     """
51 |     if x < n:
52 |         return f(x)
53 |     else:
54 |         # Формула стыковки: сдвигаем g(x) так, чтобы в точке n она совпала с f(n)
55 |         return g(x) - g(n) + f(n)
56 |
57 |
58 | if __name__ == "__main__":
59 |     import doctest
60 |
61 |     # Запуск встроенных тестов
62 |     doctest.testmod(verbose=True)
```

Рисунок 4. HTML-отчет

5. ВИЗУАЛИЗАЦИЯ ГРАФИКОВ

Для анализа поведения функции были построены графики для двух разных значений порога n :

- на графиках видны вертикальные асимптоты в точках $x=3$ и $x=-3$;
- красной пунктирной линией обозначена граница переключения функций n .

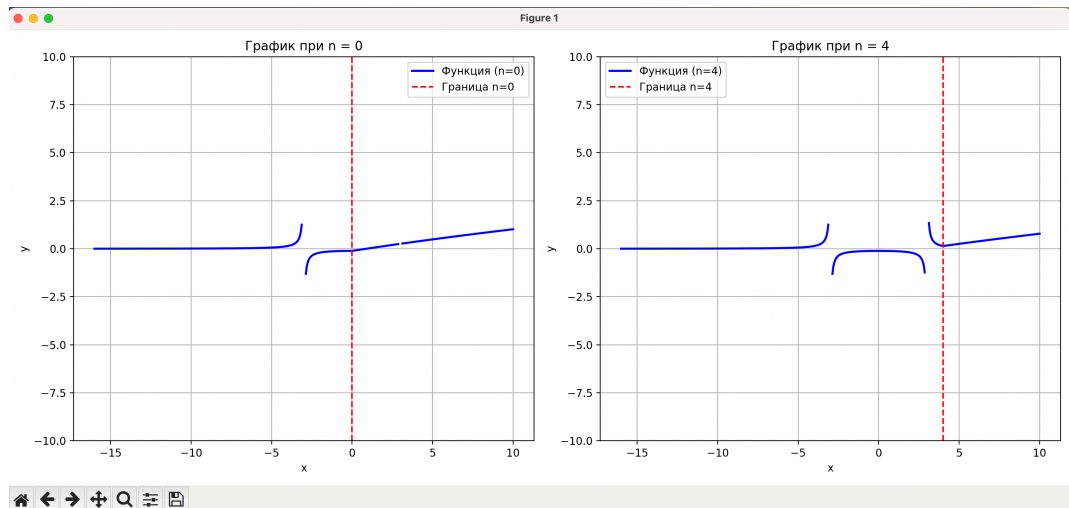


Рисунок 5. Визуализация графиков

ВЫВОДЫ

В ходе работы была реализована сдвоенная функция согласно варианту технического задания. Проведено тестирование двумя способами (doctest, unittest).

- Достигнуто полное покрытие кода тестами.
- Обработаны исключительные ситуации (деление на ноль, выход за область определения корня).
- Построены графики, подтверждающие математическую природу функций.